CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 7: Dynamic task decomposition

> **Exam objective:** Decompose complex tasks into dynamically generated subtasks that adapt as new information is discovered, rather than executing a fixed sequence regardless of intermediate findings.

Q1.

A code-audit workflow always executes ten predefined steps, including database migration checks. During a new engagement, the initial scan shows the repository contains no database layer at all. Which redesign best aligns with dynamic task decomposition?

A. Keep all ten steps so every audit is identical B. Skip irrelevant migration checks and generate new subtasks based on the discovered architecture C. Run the same fixed checklist twice to verify the result D. Replace the audit with a single summarization prompt

Answer: B **Explanation:** Dynamic decomposition adapts the work plan to observed facts, removing irrelevant branches and creating new ones where needed. A fixed checklist preserves predictability, but it wastes effort and ignores discovered context.

Q2.

A planner begins a monorepo review assuming one deployable service. Midway through, it discovers three independently released services with separate CI pipelines. What should the planner do next?

A. Continue the original one-service review so the workflow stays deterministic B. Spawn service-specific subtasks and update the plan to reflect the newly discovered topology C. Abort the run because the initial assumption was wrong D. Ignore the CI pipelines and review only root-level files

Answer: B **Explanation:** New structural information should trigger replanning and decomposition into subtasks that match the actual system shape. Continuing with the obsolete assumption would under-scope the review and miss service-specific risk.

Q3.

You are designing a research agent that must explore an unfamiliar codebase and investigate only the most suspicious areas deeply. Which control structure best supports adaptive decomposition?

A. A fixed linear script with all steps locked before execution B. A backlog or frontier of candidate subtasks prioritized as evidence is collected C. A single long prompt that asks for every possible conclusion at once D. A random sampler that chooses files without maintaining state

Answer: B **Explanation:** A prioritized task backlog lets the system add, remove, and reprioritize subtasks as evidence changes. Linear scripts and one-shot prompts do not react well to new

discoveries.

Q4.

A dynamic incident-analysis workflow keeps discovering small anomalies. The team wants to avoid endless branching while still remaining adaptive. Which stopping rule is most appropriate?

A. Stop only when no files remain anywhere in the repository B. Stop when acceptance criteria are met and no unresolved high-priority unknowns remain C. Stop after exactly twenty subtasks regardless of findings D. Stop when the first plausible root cause appears, even if conflicting evidence exists

Answer: B **Explanation:** Adaptive systems need explicit completion criteria tied to risk and unresolved uncertainty, not arbitrary step counts. Stopping at the first plausible cause or scanning everything indiscriminately both produce poor reliability.

Q5.

During a security review, a broad scan unexpectedly finds hard-coded production credentials in one module. What is the best example of dynamic decomposition?

A. Finish all low-risk checklist items before looking closer at the credentials B. Reprioritize around credential scope, exposure paths, and remediation subtasks triggered by the discovery C. Ignore the finding until the final summary phase D. Restart the entire review from the beginning with the same plan

Answer: B **Explanation:** A major finding should reshape the task graph immediately, because it changes risk and likely impacts the investigation path. Dynamic decomposition is valuable precisely when new evidence should outrank the original checklist order.

Q6.

An adaptive planner is thrashing because every minor observation causes a full replan. Which design change best preserves dynamic behavior without excessive churn?

A. Ban all replanning after the first task starts B. Replan only when changes cross defined significance thresholds or invalidate current assumptions C. Convert the system to manual-only orchestration D. Increase worker count so replans happen faster

Answer: B **Explanation:** Effective adaptive systems distinguish material discoveries from noise so they can replan when it matters. Disabling replanning entirely defeats the objective, while extra workers do not solve decision instability.

Q7.

A compliance team asks why a decomposition engine created a new subtask to review data-retention code. Which implementation best supports that auditability requirement?

A. Store each spawned subtask with the evidence and rule that caused it to be created B. Keep only the final answer to avoid extra storage C. Delete intermediate planning decisions after each phase to save space D. Use hidden prompts so users cannot question the decomposition logic

Answer: A **Explanation:** Adaptive decomposition should be explainable, especially in compliance settings, so each branch should record the trigger and rationale. Keeping only the final answer makes the behavior opaque and hard to validate.

Q8.

A repository triage system must choose between scanning every directory in a fixed order or maintaining a queue of discovered hotspots that can expand into more work. Which approach better satisfies the objective?

A. Fixed-order scanning because it is easier to script B. Hotspot-driven queued subtasks because discoveries can change what deserves attention next C. Manual note-taking by a human reviewer only D. Sorting directories alphabetically and never revisiting priorities

Answer: B **Explanation:** A hotspot queue reflects adaptive decomposition because it allows deeper investigation where evidence accumulates. Alphabetical or fixed-order processing optimizes predictability, not discovery-driven coverage.

Q9.

Two worker agents return incompatible conclusions about why a build is failing. What should a dynamic decomposition engine do?

A. Pick the first answer received and continue unchanged B. Create a reconciliation subtask that gathers additional evidence to resolve the conflict C. Discard both answers and declare the build issue untriageable D. Freeze all future decomposition to avoid inconsistency

Answer: B **Explanation:** Conflicting outputs are new information that should generate targeted follow-up work. Adaptive decomposition adds a conflict-resolution branch rather than blindly trusting arrival order.

Q10.

A planner initially assumes a bug is isolated to one service. A later trace shows the error originates in a shared library used by six services. Which response is best?

A. Keep downstream service subtasks unchanged because they were already scheduled B. Cancel or downgrade obsolete service-specific tasks and generate shared-library investigation subtasks C. Ignore the trace because it arrived after planning completed D. Force every worker to finish its original assignment before any replanning occurs

Answer: B **Explanation:** Once a core assumption is disproved, dependent tasks should be revised to match the new understanding. Continuing obsolete work wastes budget and delays the more relevant shared-library analysis.

Q11.

You have a tight token and time budget for architecture review across a very large codebase. Which decomposition strategy is most appropriate?

A. Deeply inspect every subsystem before deciding what matters B. Start with a broad cheap scan, then expand only into areas flagged by the initial evidence C. Use a fixed sequence of expensive tasks for consistency D. Defer all branching decisions until the very end

Answer: B **Explanation:** Dynamic decomposition is especially useful under budget constraints because it supports coarse-to-fine exploration. Spending heavily everywhere defeats the purpose of evidence-driven prioritization.

Q12.

A planner is generating hundreds of tiny subtasks such as `read one file` and `read next file`, causing orchestration overhead to exceed useful work. What is the best improvement?

A. Increase decomposition granularity thresholds so related micro-steps are grouped into meaningful units B. Disable all logging so the tiny tasks seem faster C. Add more coordinators to create even smaller tasks in parallel D. Convert every file read into a separate approval request

Answer: A **Explanation:** Adaptive planning still needs sensible task granularity; otherwise, orchestration cost overwhelms execution value. Grouping related steps preserves adaptability while reducing needless management overhead.

Q13.

A search workflow first classifies modules by technology stack, then discovers independent clusters requiring different expert agents. How should dynamic decomposition handle the next phase?

A. Continue with one generic worker because branching adds complexity B. Generate specialized subtasks per cluster and run independent ones in parallel C. Re-run the classification step repeatedly until no new stacks are found D. Collapse all clusters into a single summary prompt immediately

Answer: B **Explanation:** Once distinct clusters are identified, decomposition should branch into targeted work streams matched to each cluster. That keeps the investigation adaptive while exploiting independence for efficiency.

Q14.

Which metric best indicates that a dynamic decomposition system is working as intended during an exploratory troubleshooting task?

A. It completes the originally written step list without deviation B. It resolves the most relevant discovered branches while avoiding unnecessary work on disproven paths C. It always creates the same number of subtasks for every repository D. It never changes task priority after execution begins

Answer: B **Explanation:** Successful adaptive decomposition responds to evidence and closes the branches that matter most. Rigid adherence to the initial list is actually a sign that the system is not adapting.

Q15.

A log-analysis agent starts with a broad request to diagnose intermittent failures. It quickly identifies three distinct anomaly clusters: authentication spikes, queue backlogs, and cache invalidations. What is the best next action?

A. Force one worker to finish all three clusters serially using the original prompt B. Create cluster-specific subtasks and let each branch gather evidence appropriate to that anomaly type C. Ignore the clusters and continue scanning logs chronologically only D. Stop immediately because multiple hypotheses exist

Answer: B **Explanation:** Distinct anomaly clusters are exactly the kind of discovery that should drive new targeted subtasks. Adaptive branching improves both coverage and root-cause accuracy without requiring a fixed serial script.